



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Muhammad Luthfi Hilmy
NPM	2428240032
Kelas	SI5B
Topik Tugas 1	PMI Stok Darah
Entitas utama	blood-stocks
Tautan repositori	https://github.com/fiyoraa/si5b_kuis1_muhammadluthfihilmy
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	Kamis, 8 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	PMI Stok Darah
Entitas utama (tunggal)	blood stock
Alamat dasar	/blood-stocks
Field data	id, golonganDarah, rhesus, jumlahKantong, lokasi, tanggalUpdate
Field wajib saat POST	golonganDarah, rhesus, jumlahKantong, lokasi, tanggalUpdate
Query string untuk filter	golonganDarah
Data awal	18 data stok darah PMI
Modul lain (jika ada)	logger, cekApiKey, errorHandler

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/blood-stocks	Tidak	200	Mengambil seluruh data dan filter
2	GET	/blood-stocks/:id	Tidak	200	Mengambil satu data
3	POST	/blood-stocks	Ya	201	Menambah data
4	PUT	/blood-stocks/:id	Ya	200	Mengubah data
5	DELETE	/blood-stocks/:id	Ya	204	Menghapus data

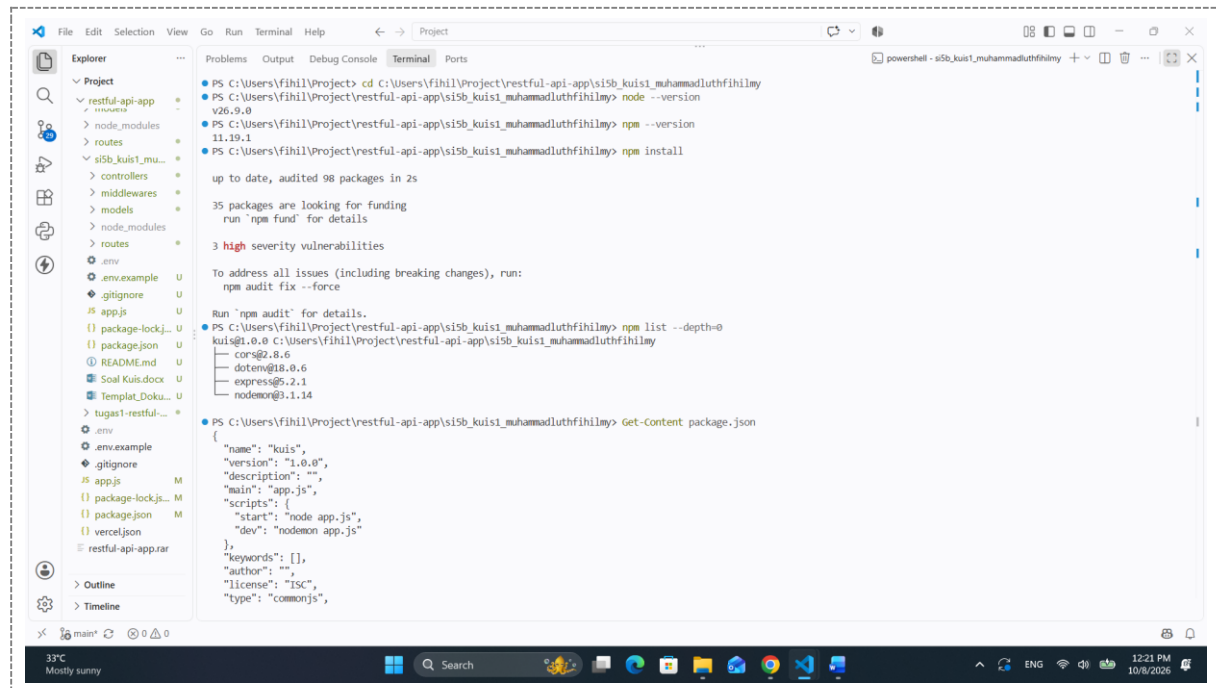
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

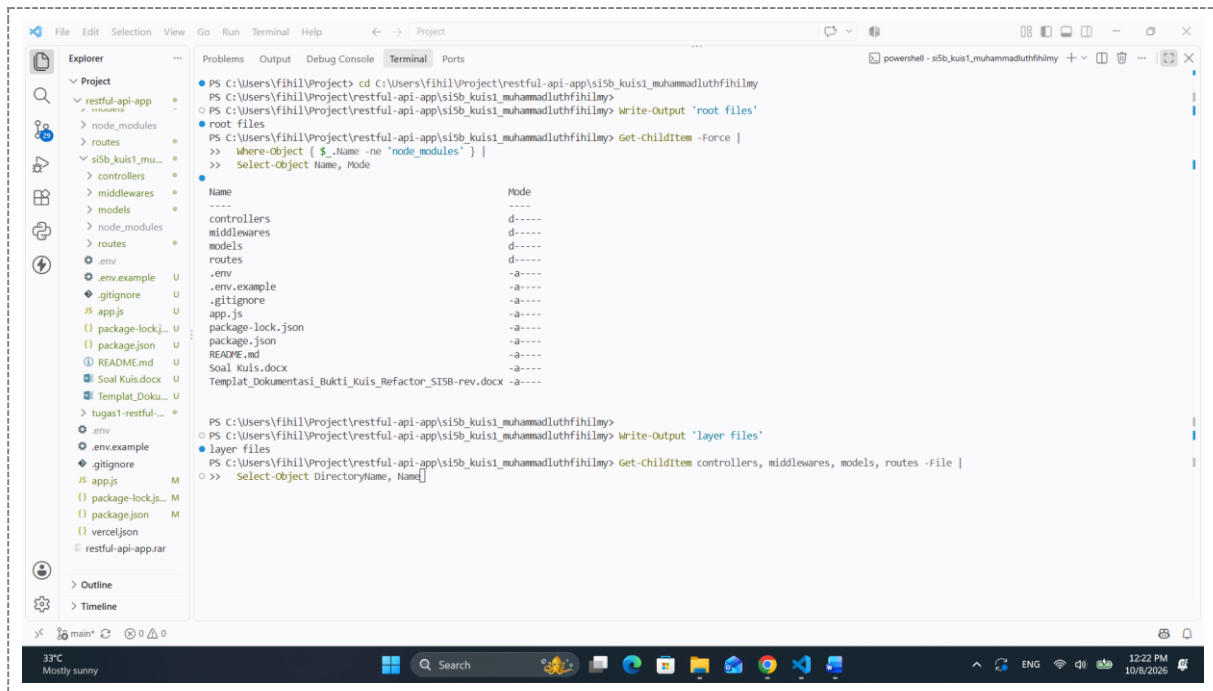
Project kuis dibuat menggunakan Node.js dengan Express.js sebagai framework utama untuk membangun RESTful API. Dependency cors digunakan untuk mengatur akses lintas origin, sedangkan dotenv digunakan untuk membaca konfigurasi dari file `.env`. Nodemon dipasang sebagai dev dependency agar server dapat dimuat ulang secara otomatis ketika terdapat perubahan kode. Instalasi dilakukan di dalam folder project `si5b_kuis1_muhammadruthifihilm`.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Struktur project dipisahkan menjadi beberapa layer agar setiap bagian memiliki tanggung jawab yang jelas. Folder models mengelola data dan operasi CRUD. Folder controllers menangani request, validasi, dan response. Folder routes menghubungkan URL dengan controller menggunakan `express.Router()`. Folder middlewares berisi fungsi umum untuk mencatat request, memeriksa API key, dan menangani error. File `app.js` berfungsi sebagai penggabung seluruh komponen aplikasi.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

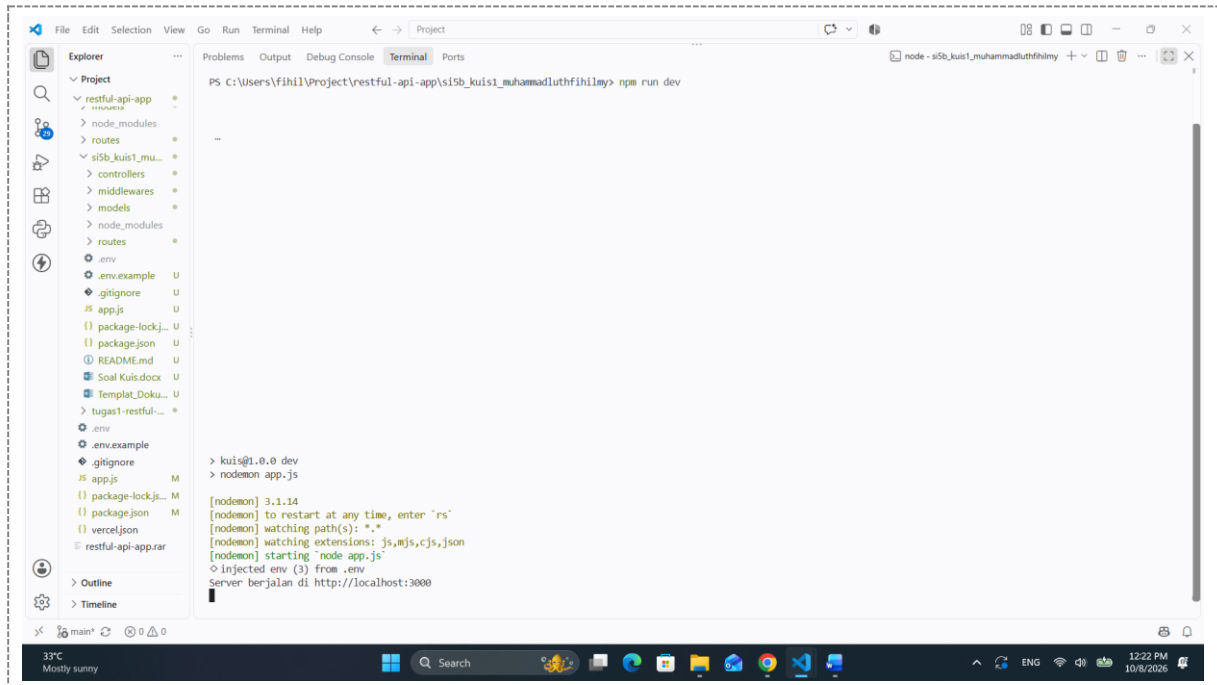
Berkas	Penjelasan Anda
models/bloodStocksModel.js	Menyimpan 18 data awal dan fungsi CRUD tanpa menggunakan req atau res
controllers/bloodStocksController.js	Menangani request, membaca parameter/query/body, melakukan validasi, memanggil model, dan mengirim response

Berkas	Penjelasan Anda
routes/bloodStocksRoutes.js	Memetakan endpoint /blood-stocks ke fungsi controller menggunakan <code>express.Router()</code>
middlewares/logger.js	Mencatat waktu, method, dan URL setiap request yang masuk.
middlewares/cekApiKey.js	Memeriksa header <code>x-api-key</code> sebelum POST, PUT, dan DELETE diteruskan ke controller.
middlewares/errorHandler.js	Menangani JSON rusak, endpoint yang tidak ditemukan, dan error server dalam format JSON.
app.js	Memuat environment, memasang CORS dan middleware global, mendaftarkan route, serta menjalankan server.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Server dijalankan menggunakan script `npm run dev` yang menjalankan Nodemon dan file `app.js`. Nomor port dibaca dari `process.env.PORT` pada file `.env`, yaitu port 3000. Jika variabel `PORT` tidak tersedia, aplikasi menggunakan nilai cadangan 3000. Apabila port 3000 sedang digunakan aplikasi lain, server tidak dapat memakai port tersebut sehingga proses lama perlu dihentikan atau nilai port pada `.env` diubah.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

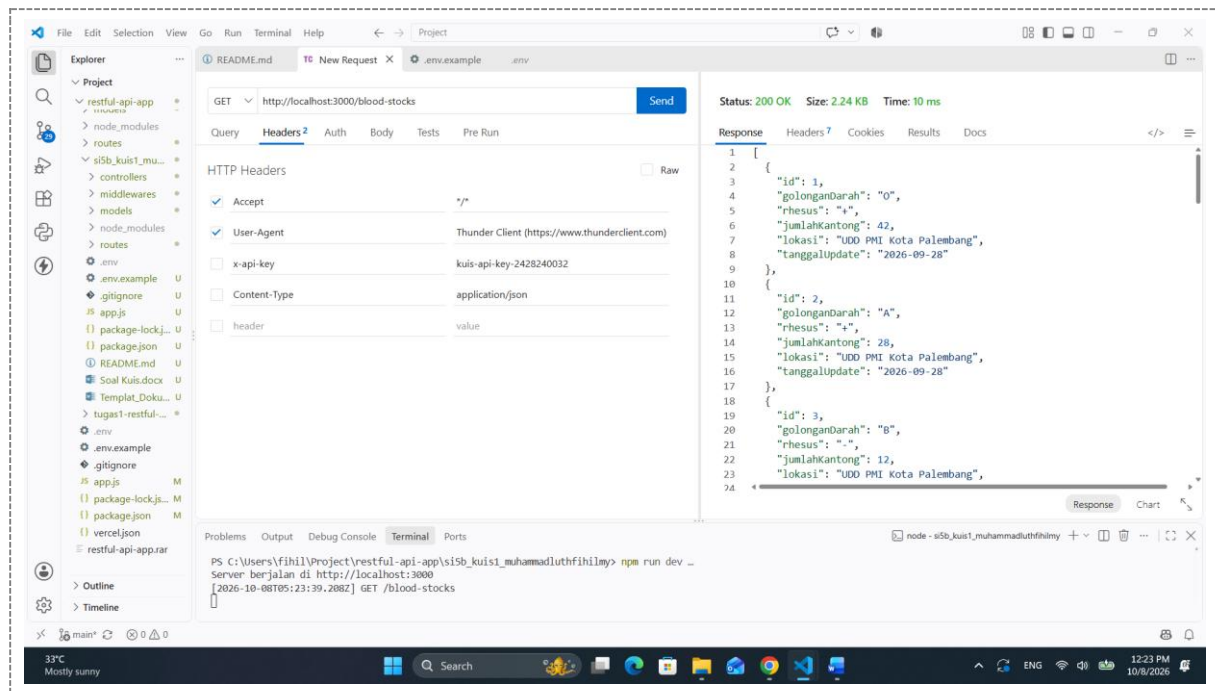
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /blood-stocks
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK; data stok darah berhasil ditampilkan dalam bentuk larik JSON.
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

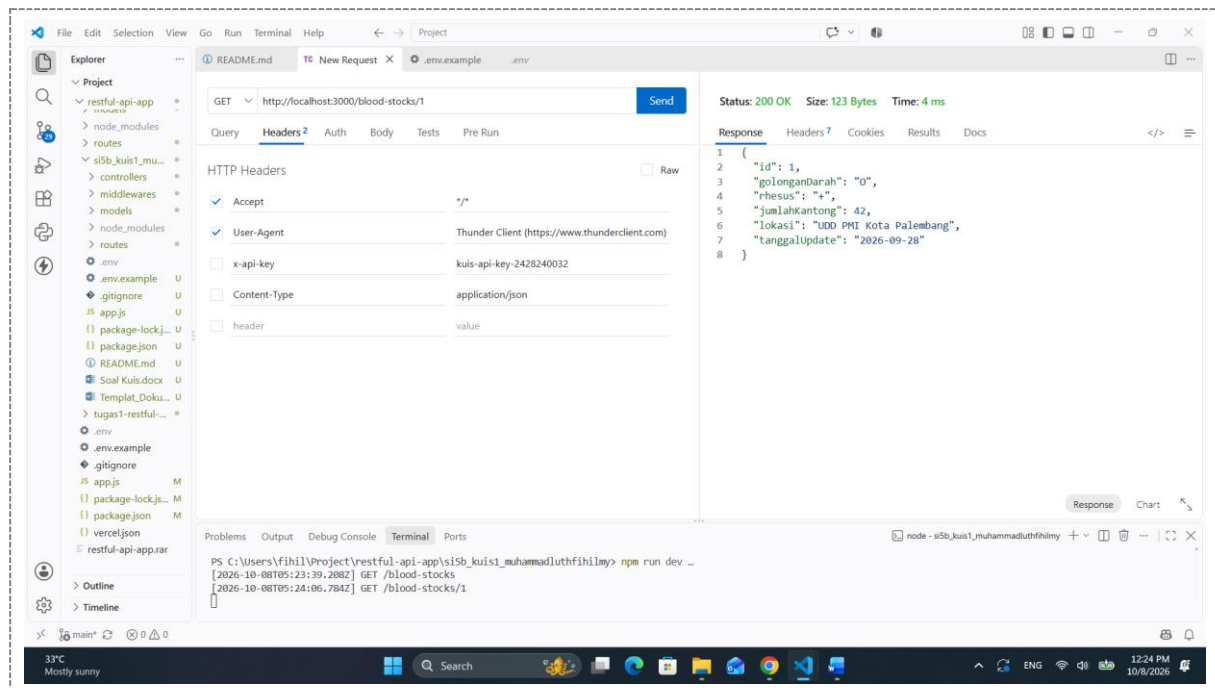
Penjelasan:

Request GET diterima oleh route `/blood-stocks`, kemudian diteruskan ke fungsi `getAll` pada controller. Controller memanggil fungsi `getAll` pada model untuk mengambil seluruh data stok darah. Data kemudian dikirimkan langsung sebagai array JSON dengan status 200 OK.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET <code>/blood-stocks/1</code>
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK; data dengan id 1 berhasil ditemukan dan ditampilkan.
Lulus?	[✓] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

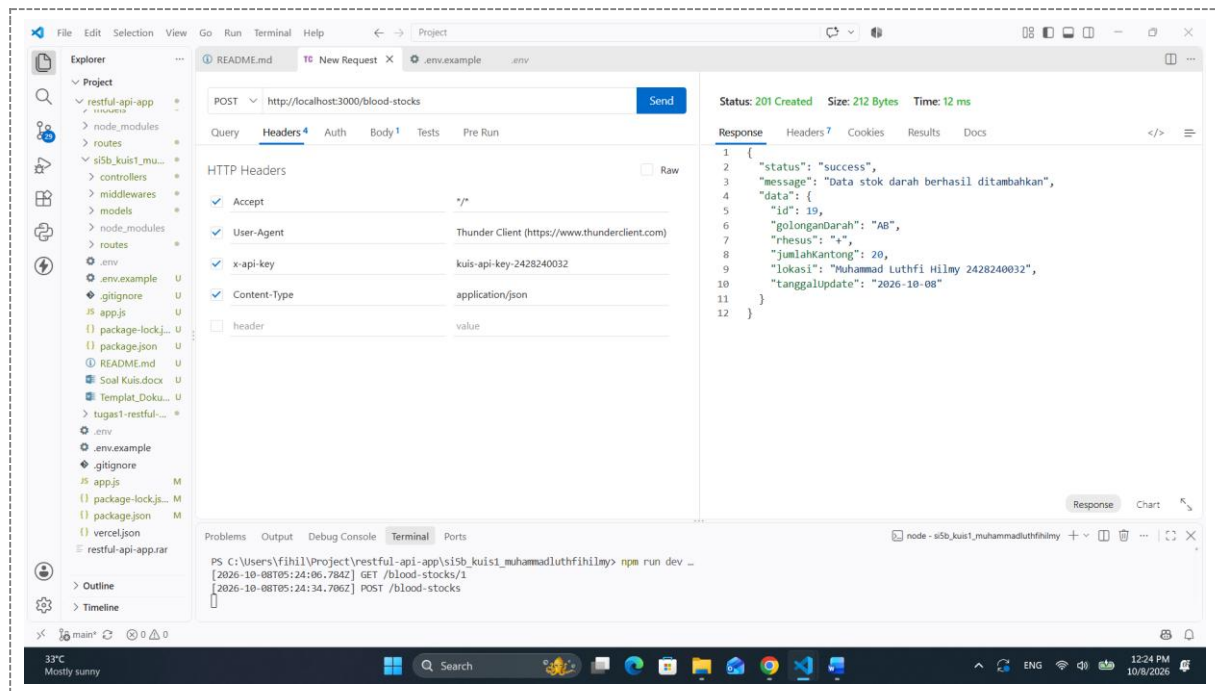
Penjelasan:

ID diambil dari req.params.id, kemudian diubah menjadi angka agar dapat dibandingkan dengan ID yang tersimpan pada model. Controller memanggil fungsi getByID pada model untuk mengambil satu data berdasarkan ID.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /blood-stocks
Header	x-api-key: kuis-api-key-2428240032 Content-Type: application/json
Body	{ "golonganDarah": "AB", "rhesus": "+", "jumlahKantong": 20, "lokasi": "Muhammad Luthfi Hilmy 2428240032", "tanggalUpdate": "2026-10-08" }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created; data berhasil ditambahkan dengan id 19.
Lulus?	[✓] Ya [] Tidak



Gambar 6. Hasil POST data baru

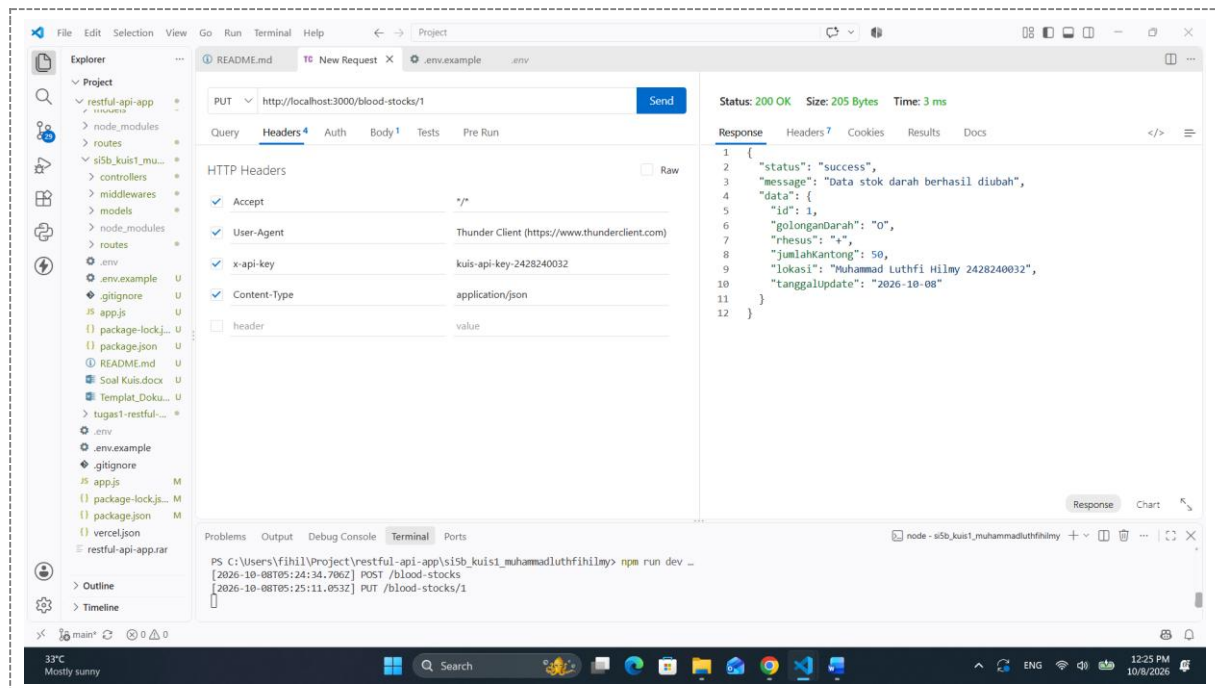
Penjelasan:

Middleware cekApiKey memeriksa header x-api-key sebelum request diteruskan ke controller. Middleware express.json() membaca body JSON menjadi req.body. Controller melakukan validasi, lalu memanggil model untuk membuat data baru dengan ID otomatis. Status 201 Created digunakan karena resource baru berhasil dibuat.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /blood-stocks/1
Header	x-api-key: kuis-api-key-2428240032 Content-Type: application/json
Body	{ "golonganDarah": "O", "rhesus": "+", "jumlahKantong": 50, "lokasi": "Muhammad Luthfi Hilmy 2428240032", "tanggalUpdate": "2026-10-08" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK; data dengan id 1 berhasil diubah.
Lulus?	[✓] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

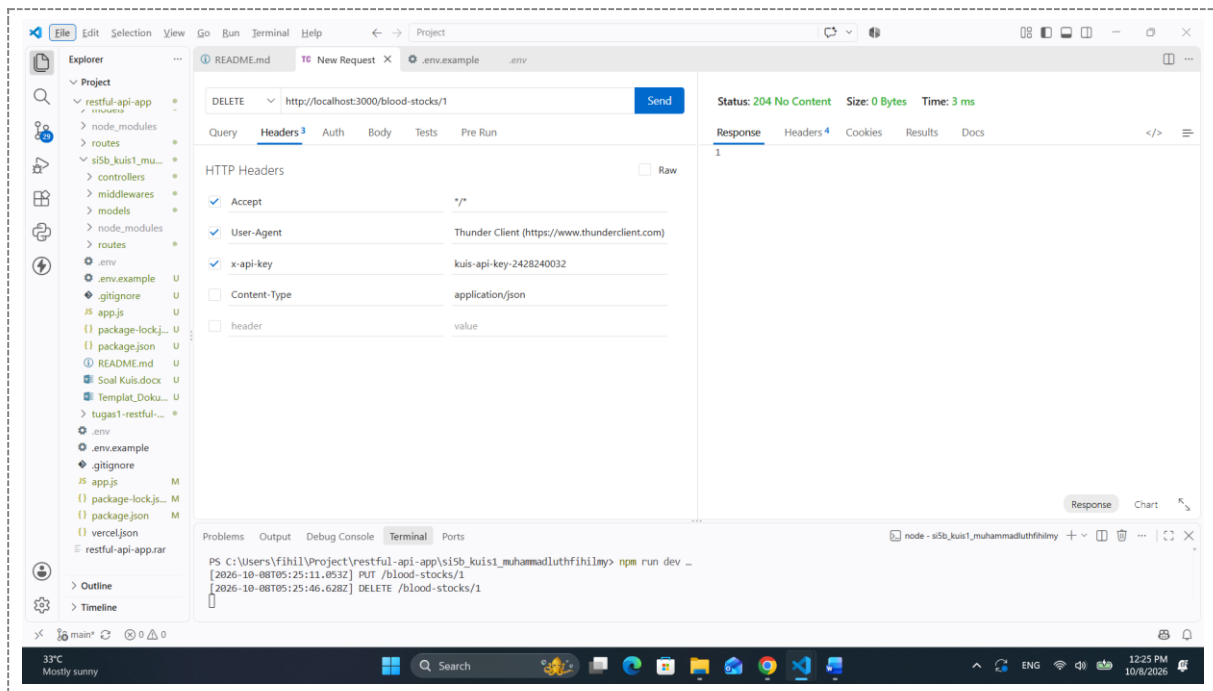
Penjelasan:

Request PUT dilindungi oleh middleware cekApiKey. Controller mengambil ID dari parameter route, memvalidasi seluruh field wajib, lalu memanggil fungsi update pada model. Data lama dengan ID 1 diganti menggunakan data baru dari request. Status 200 OK menunjukkan bahwa pembaruan berhasil.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE //blood-stocks/1
Header	x-api-key: kuis-api-key-2428240032
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content; data dengan id 1 berhasil dihapus dan respons tidak memiliki body.
Lulus?	[✓] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Request DELETE dilindungi oleh middleware cekApiKey. Controller mencari ID melalui model, kemudian model menghapus data menggunakan fungsi remove. Status 204 No Content berarti data berhasil dihapus dan server tidak mengirimkan isi response

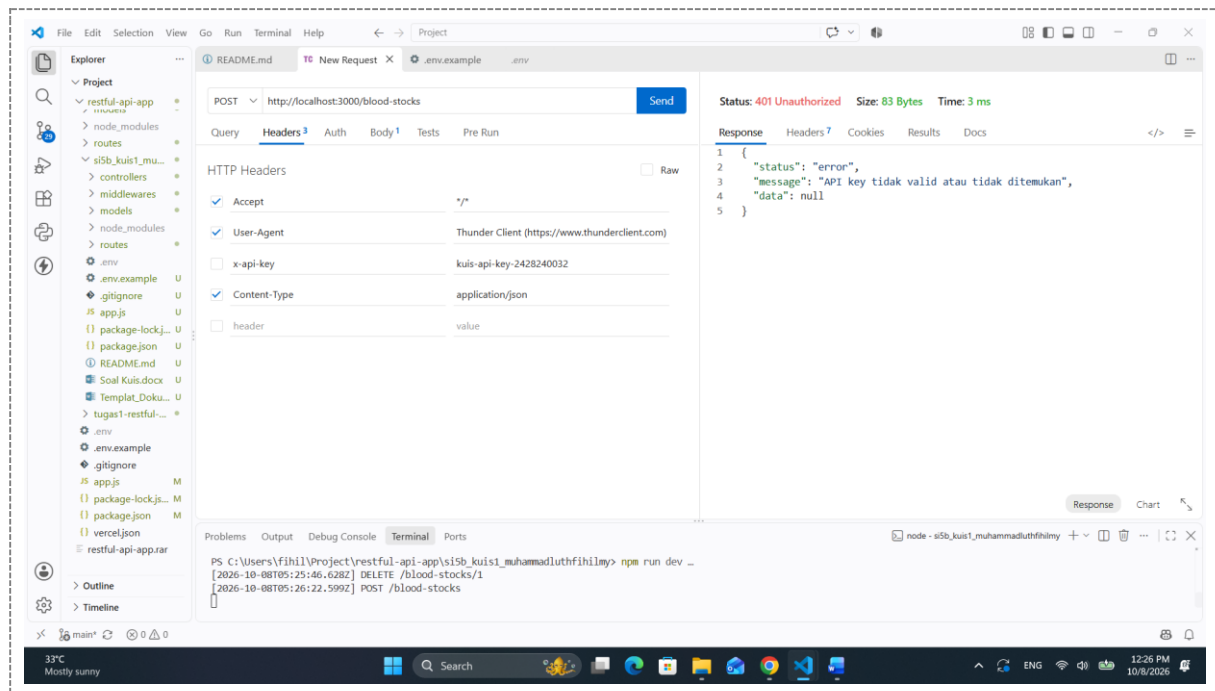
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /blood-stocks
Header	Content-Type: application/json
Body	<pre>{ "golonganDarah": "AB", "rhesus": "+", "jumlahKantong": 20, "lokasi": "Muhammad Luthfi Hilmy 2428240032", "tanggalUpdate": "2026-10-08" }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized; API key tidak valid atau tidak ditemukan.
Lulus?	☒ Ya ☐ Tidak



Gambar 9. POST tanpa API key ditolak

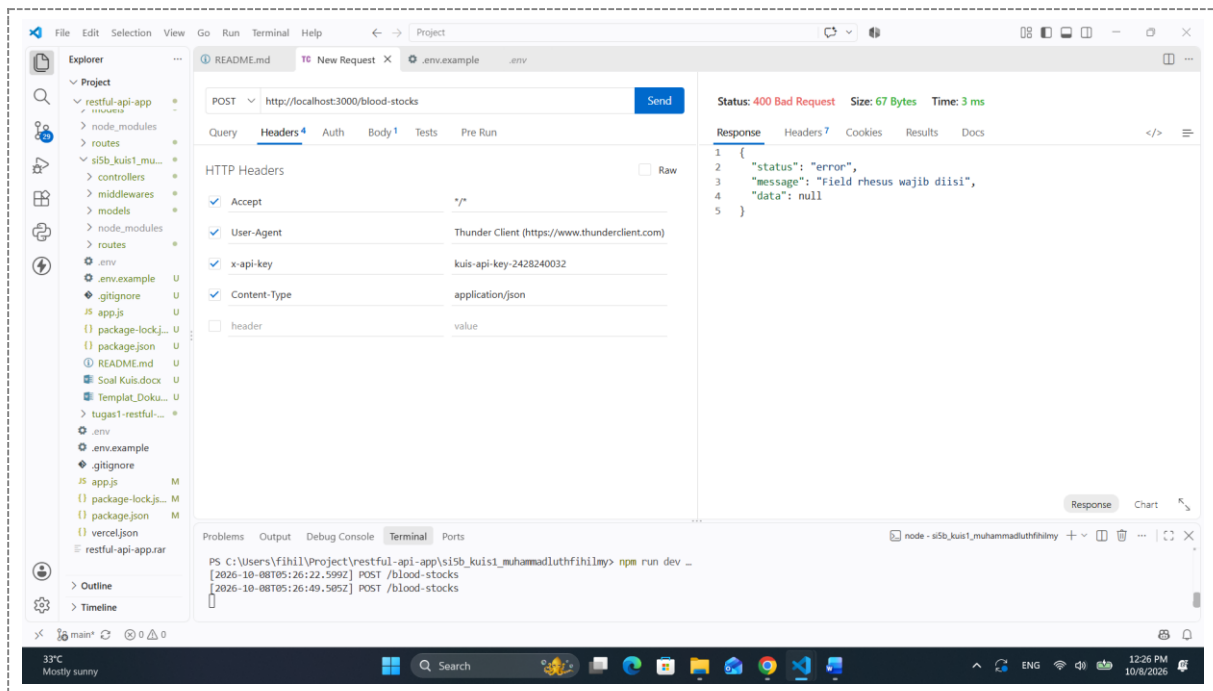
Penjelasan:

Request dihentikan oleh middleware cekApiKey karena header x-api-key tidak dikirimkan. Controller tidak sempat dijalankan sehingga data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /blood-stocks
Header	x-api-key: kuis-api-key-2428240032 Content-Type: application/json
Body	{ "golonganDarah": "O" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request; Field rhesus wajib diisi.
Lulus?	[✓] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

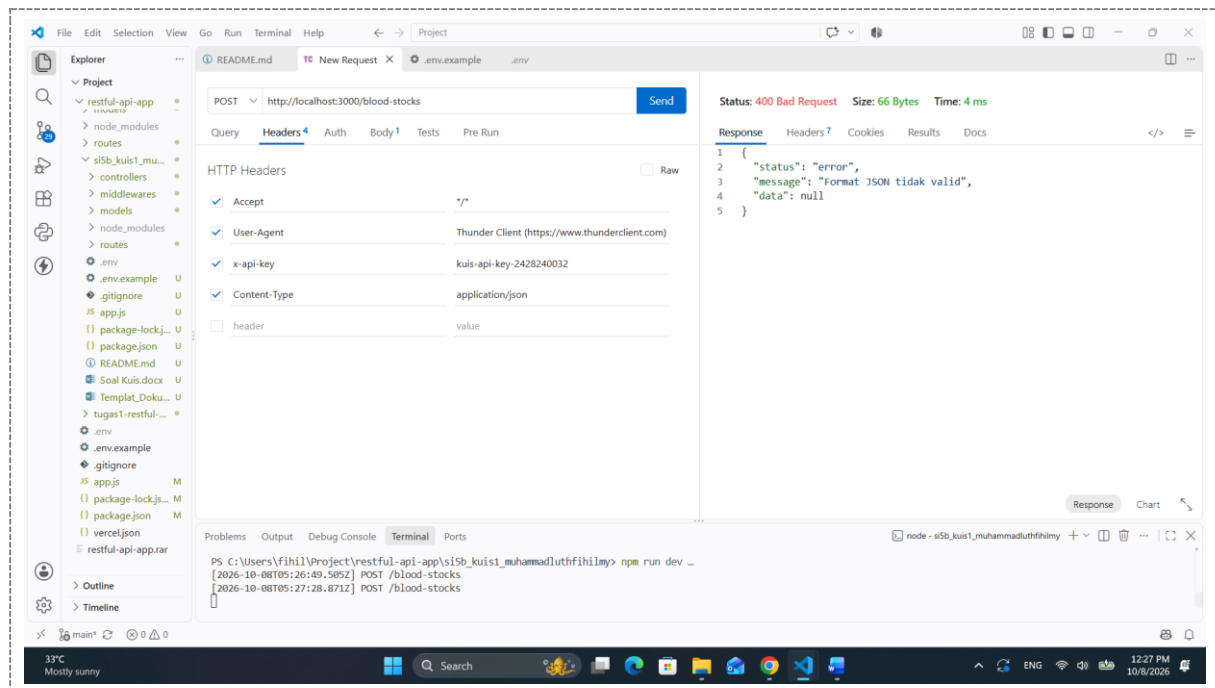
Penjelasan:

API key sudah benar sehingga request diteruskan ke controller. Controller menjalankan validasi terhadap seluruh field wajib. Karena body tidak lengkap, request ditolak dengan status 400 Bad Request dan data tidak ditambahkan.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /blood-stocks
Header	x-api-key: kuis-api-key-2428240032 Content-Type: application/json
Body	{ "golonganDarah": "O", }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request; Format JSON tidak valid.
Lulus?	[✓] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

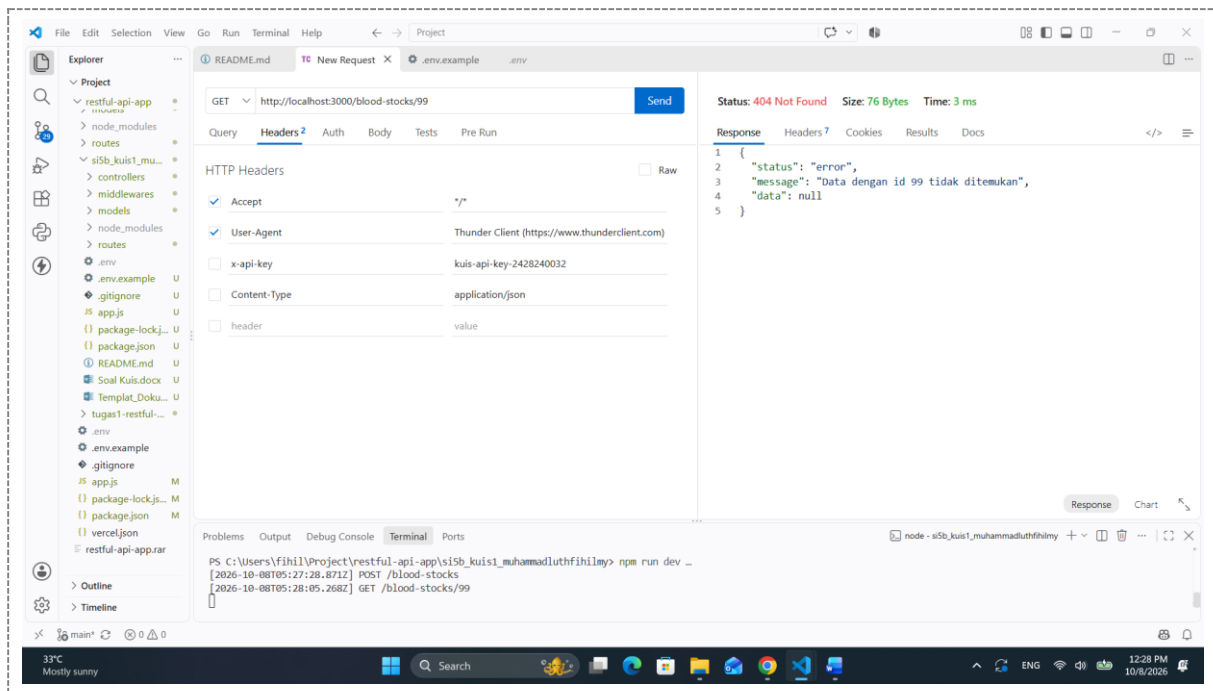
Penjelasan:

JSON rusak dideteksi oleh middleware `express.json()` sebelum controller dijalankan. Error diteruskan ke errorHandler, kemudian diubah menjadi response JSON dengan status 400. Dengan demikian, server tidak mengirim halaman HTML bawaan Express.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /blood-stocks/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found; Data dengan id 99 tidak ditemukan.
Lulus?	[✓] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Controller membaca ID 99 dari req.params.id, lalu memanggil fungsi getById pada model. Karena data dengan ID tersebut tidak ditemukan, controller meneruskan error 404 ke errorHandler. Middleware tersebut mengirimkan response JSON dengan status 404 Not Found.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Ya
T02	Menampilkan satu data berdasarkan ID	200	200	Ya
T03	Menambah data baru	201	201	Ya
T04	Mengubah data	200	200	Ya
T05	Menghapus data	204	204	Ya
N01	Menambah data tanpa API key	401	401	Ya
N02	Menambah data dengan body tidak lengkap	400	400	Ya
N03	Mengirim JSON yang rusak	400	400	Ya
N04	Mengambil ID yang tidak ada	404	404	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/fiyoraa/si5b_kuis1_muhammad_luthfihilmy
Pesan commit terakhir	Kuis 1 id commit: 430f713e820c106bb67986ce3512228aa0790d73
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Pada project ini, Route bertugas memetakan alamat dan method HTTP ke fungsi controller. Route tidak menangani proses bisnis atau penyimpanan data. Contohnya pada file `routes/bloodStocksRoutes.js`: `router.get('/:id', bloodStocksController.getById);`

Baris tersebut mengarahkan request GET `/blood-stocks/:id` ke fungsi `getById` pada controller.

Controller bertugas menerima request, membaca `req.params`, `req.query`, atau `req.body`, melakukan validasi, memanggil model, dan mengirimkan response. Contohnya pada file `controllers/bloodStocksController.js`: `const stock = id === null ? null : bloodStocksModel.getById(id);`

Baris tersebut menunjukkan controller memanggil fungsi `getById` dari model setelah mengambil ID dari request.

Model bertugas menyimpan data dan menjalankan operasi pengolahan data seperti mengambil, menambah, mengubah, dan menghapus data. Model tidak menggunakan `req` atau `res`. Contohnya pada file `models/bloodStocksModel.js`: `return bloodStocks.find((stock) => stock.id === id);`

Baris tersebut mencari satu data stok darah berdasarkan ID.

Dengan pembagian tersebut, setiap layer memiliki tanggung jawab yang jelas. Route mengatur alamat, controller mengatur alur request dan response, sedangkan model mengatur data.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan kesalahan menjadi lebih konsisten dan mengurangi pengulangan kode pada setiap route. Controller cukup meneruskan error menggunakan `next(error)`, kemudian semua error diproses oleh satu fungsi errorHandler. Contoh pada file `controllers/bloodStocksController.js`: `return next(errorHttp(404, `Data dengan id ${req.params.id} tidak ditemukan`));`

Error tersebut diproses oleh errorHandler pada file `middlewares/errorHandler.js`:

```
res.status(status).json({
  status: 'error',
  message: error.message,
  data: null,
});
```

Keuntungan lainnya adalah semua response error memiliki format JSON yang sama. Error JSON rusak juga dapat ditangani secara terpusat:

```
if (error.type === 'entity.parse.failed') {
  return res.status(400).json({
    status: 'error',
    message: 'Format JSON tidak valid',
    data: null,
  });
}
```

Bukti dari pengujian:

1. saat mengakses GET `/blood-stocks/99` server mengembalikan: 404 Not Found dengan respon:

```
{
  "status": "error",
  "message": "Data dengan id 99 tidak ditemukan",
  "data": null
}
```

2. saat mengakses json rusak pada POST `/blood-stocks` server mengembalikan: 400 Bad Request dengan respon:

```
{
  "status": "error",
  "message": "Format JSON tidak valid",
  "data": null
}
```

3. Saat mengakses endpoint yang tidak terdaftar, misalnya: GET `/endpoint-salah` middleware `notFoundHandler` mengembalikan:

```
{
  "status": "error",
  "message": "Endpoint tidak ditemukan",
  "data": null
}
```

```
}
```

Jadi, error handler terpusat memastikan error dari beberapa sumber memiliki format response yang seragam, yaitu status, message, dan data, serta memudahkan pemeliharaan kode dibandingkan menulis penanganan error berulang kali di setiap route.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	Ya
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100